

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Web Client and Backend Development Framework.

Reporte de Integración BackEnd y FrontEnd con Spring Security y
Persistencia en Base de Datos

Profesor: M. en C. José Asunción Enríquez Zárate

Alumno: Luis David Martínez Martínez

ld.martinez.117@gmail.com

7CM3

22 de Junio, 2026

Contents

List of Figures

List of Tables

1 Introducción

La integración de arquitecturas web modernas se basa habitualmente en el desacoplamiento de capas, donde el Back End opera como un proveedor de servicios RESTful autónomo y el Front End actúa como un cliente enriquecido (Single Page Application - SPA) que gestiona la experiencia del usuario y las vistas.

Para lograr que esta comunicación sea segura, el Back End debe contar con un framework de seguridad robusto como Spring Security, configurando filtros de autenticación y autorización capaces de validar el acceso según roles de usuario persistidos en base de datos. En esta práctica se documenta la finalización e integración del módulo de reservaciones del sistema hotelero, así como las estrategias aplicadas para establecer un canal seguro entre ambas capas.

2 Objetivo de la práctica

Completar el módulo de reservaciones y realizar la integración del Front End con los servicios expuestos por el Back End en Spring Boot, asegurando las peticiones mediante Spring Security con usuarios y roles cargados desde una base de datos relacional MySQL.

3 Conceptos

3.1 *Autenticación y Autorización*

La autenticación valida la identidad de un usuario (¿quién es?), típicamente cotejando credenciales cifradas en base de datos. La autorización determina los privilegios del usuario autenticado (¿qué recursos puede acceder?) comparando sus roles con las reglas de negocio establecidas.

3.2 *BCrypt y Hashing de Contraseñas*

BCrypt es una función de hashing de contraseñas adaptable y segura que incorpora un factor de costo y un valor salt aleatorio por cada password, impidiendo ataques de fuerza bruta o diccionario basados en tablas Rainbow.

3.3 *Políticas CORS (Cross-Origin Resource Sharing)*

Mecanismo de seguridad de los navegadores web que restringe a las páginas realizar peticiones HTTP hacia un dominio diferente al que originó la página original. El Back End debe estar configurado explícitamente para admitir cabeceras y métodos HTTP provenientes de los puertos del Front End.

4 Descripción del sistema

La aplicación web, denominada **ReservaEasy**, gestiona el flujo de reserva de habitaciones de hotel. Ofrece soporte para dos perfiles:

1. **Administradores (ROLE_ADMIN):** Crean, modifican y eliminan habitaciones, y gestionan las reservaciones globales de todo el hotel.
2. **Clientes (ROLE_USER):** Consultan cuartos disponibles, registran reservaciones a su nombre y cancelan o modifican sus reservaciones.

5 Modelo de base de datos

La base de datos relacional almacena la información organizada bajo las entidades descritas en la Tabla ??.

Tabla	Propósito	Atributos Principales
rooms	Catálogo de habitaciones	id, tipo, numero, precio
usuarios	Datos y acceso de usuarios	id, username, password, email
roles	Listado de roles autorizados	id, nombre (ROLE_USER, ROLE_ADMIN)
usuario_rols	Tabla asociativa N:M	usuario_id, rol_id
reservaciones	Registro de reservas	id, cuarto_id, usuario_id, estado

Table 1: Relaciones y tablas en base de datos MySQL

6 Desarrollo e Integración

6.1 Configuración de Seguridad en Spring Boot

En SecurityConfig.java se inyecta el codificador BCrypt y se restringen los métodos HTTP basados en los roles persistidos:

```
1 @Configuration
2 @EnableWebSecurity
3 public class SecurityConfig {
4     @Bean
5     public PasswordEncoder passwordEncoder() {
6         return new BCryptPasswordEncoder();
7     }
8
9     @Bean
10    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
11        http.cors(cors -> cors.configurationSource(corsConfigurationSource()))
12            .csrf(csrf -> csrf.disable())
13            .authorizeHttpRequests(auth -> auth
14                .requestMatchers("/api/v1/auth/**").permitAll()
15                .requestMatchers(HttpMethod.GET, "/api/v1/cuartos/**").permitAll()
16                .requestMatchers(HttpMethod.POST, "/api/v1/cuartos/**").hasRole("ADMIN")
17                .requestMatchers(HttpMethod.GET, "/api/v1/reservaciones").hasRole("ADMIN")
18                .anyRequest().authenticated()
19            )
20            .httpBasic(Customizer.withDefaults());
21        return http.build();
22    }
23 }
```

6.2 Carga de Detalles de Usuario desde la Base de Datos

Se implementó CustomUserDetailsService.java para leer credenciales de la base de datos:

```
1 @Service
2 @RequiredArgsConstructor
3 public class CustomUserDetailsService implements UserDetailsService {
4     private final UsuarioRepository usuarioRepository;
5
6     @Override
7     public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
8         Usuario usuario = usuarioRepository.findByUsername(username)
9             .orElseThrow(() -> new UsernameNotFoundException("Usuario no encontrado"));
10        return new User(
11            usuario.getUsername(),
12            usuario.getPassword(),
13            usuario.getRoles().stream()
14                .map(r -> new SimpleGrantedAuthority(r.getNombre()))
15                .collect(Collectors.toList())
16        );
17    }
18 }
```

6.3 Integración en la SPA (Front End)

En el archivo app.js del Front End se enlazan los formularios REST adjuntando las credenciales codificadas en Base64 en el encabezado de autorización HTTP:

```
1 const authHeader = 'Basic ' + btoa(username + ':' + password);
2 const rooms = await fetch('http://localhost:8090/api/v1/cuartos', {
3     headers: { 'Authorization': authHeader }
4 });
```

7 Resultados e Integración

Para comprobar la correcta integración del Back End en Spring Boot y el Front End en Angular 19, se definieron casos de prueba simulando el comportamiento del sistema según los roles del usuario autenticado. A continuación, se detallan los resultados y evidencias de cada caso de prueba.

7.1 Flujo de Trabajo del Cliente (ROLE_USER)

El usuario cliente tiene un flujo orientado a la visualización de habitaciones disponibles y a la autogestión de sus reservaciones.

1. **Autenticación y Registro:** El cliente inicia sesión a través del modal interactivo. La aplicación envía las credenciales en Base64 en la cabecera HTTP de autorización. Al confirmarse el éxito, se almacena la sesión y se habilitan dinámicamente en el menú las opciones de “Mis Reservas” y “Documentos”.

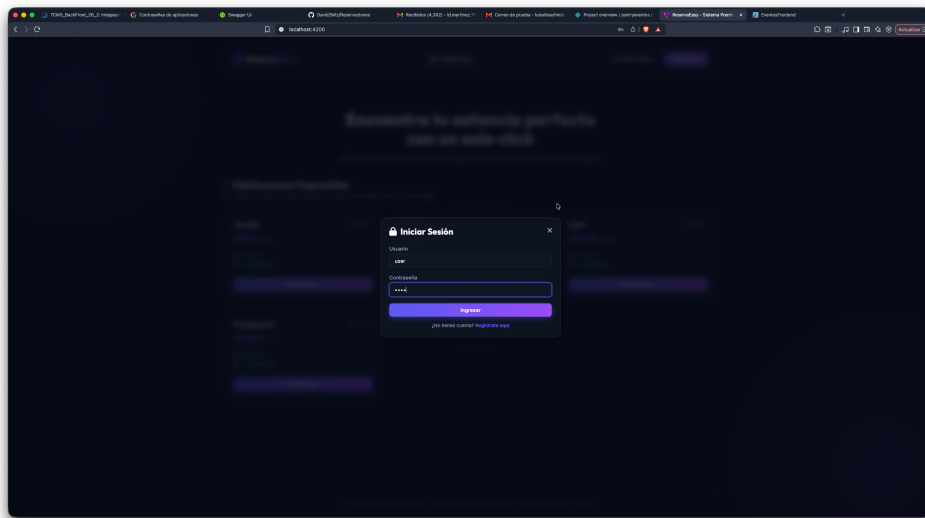


Figure 1: Interfaz de inicio de sesión y registro de usuarios en la Single Page Application.

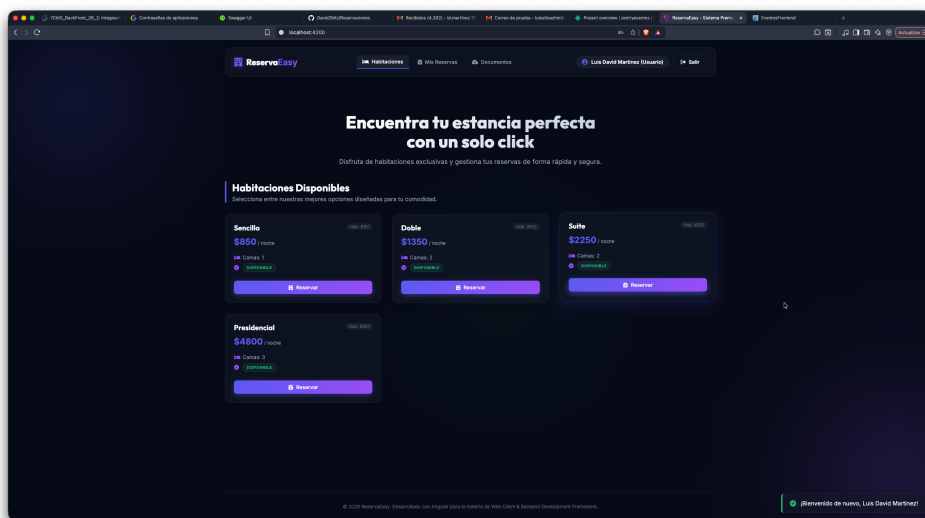


Figure 2: Inicio de sesión correcto.

2. **Inspección de Habitaciones:** El catálogo principal consulta el endpoint GET `/api/v1/cuartos`. El sistema renderiza las habitaciones en una cuadrícula responsiva, mostrando su tipo, precio, camas y estado. Si una habitación no está libre, se deshabilita automáticamente su botón de reserva.

Figure 3: Catálogo dinámico de habitaciones cargado desde la base de datos MySQL.

3. **Proceso de Reservación con Cotización Dinámica:** Al hacer clic en “Reservar”, se despliega un formulario modal. Cuando el usuario selecciona las fechas de entrada y salida, la aplicación calcula en tiempo real el costo estimado multiplicando el precio de la habitación por el número de noches. Al enviar, el Back End valida que no existan traslapes de fechas, guarda la reservación como “CONFIRMADA” y dispara la notificación por correo SMTP.

Figure 4: Modal interactivo de reservación mostrando la cotización calculada en tiempo real.

4. **Gestión de Reservaciones Propias (Historial y Cancelación):** El usuario accede a la vista “Mis Reservas” (GET /api/v1/reservaciones/usuario/{id}). Aquí visualiza sus estancias. Si el estado es “CONFIRMADA”, se muestran opciones para editar las fechas o cancelar la reserva (DELETE /api/v1/reservaciones/{id}). Al cancelar, se actualiza el estado a “CANCELADA” y envía la alerta de correo electrónico.

Figure 5: Sección de Mis Reservas del cliente, con opciones para modificar fechas o cancelar.

7.2 Flujo de Trabajo del Administrador (ROLE_ADMIN)

El usuario administrador cuenta con permisos globales sobre la plataforma para gestionar el catálogo físico de habitaciones del hotel y realizar auditorías.

1. **Administración del Inventario de Cuartos:** Al acceder con el rol de administrador, la SPA habilita la vista “Panel Admin”. A través del formulario de carga, el administrador registra nuevas habitaciones indicando precio, número y camas, o bien selecciona habitaciones existentes para editar sus características o eliminarlas de la base de datos.

Figure 6: Formulario administrativo para el control del inventario de habitaciones (CRUD).

2. **Auditoría y Cancelación Global de Reservaciones:** En la misma vista administrativa, se muestra una tabla dinámica que recopila todas las reservas realizadas por todos los usuarios del sistema. El administrador puede verificar quién ha reservado cada cuarto y tiene la facultad de cancelar cualquier reserva activa por motivos operativos.

Figure 7: Tabla de auditoría global de reservaciones del sistema para ROLE_ADMIN.

7.3 Gestión y Soporte de Archivos en la Base de Datos

Como soporte de negocio, los usuarios pueden subir archivos adjuntos (identificación oficial, comprobante de pago) y descargarlos asociándolos a su ID de archivo.

Figure 8: Módulo de carga y descarga de comprobantes persistidos en formato BLOB en MySQL.

8 Conclusión

La integración de una SPA en HTML/CSS/JS con una API robusta en Spring Boot y Spring Security demuestra la viabilidad de arquitecturas desacopladas modernas. El uso de Spring Security con usuarios y roles persistidos permite aislar la lógica de permisos en el Back End, garantizando que el sistema sea seguro incluso ante posibles intentos de vulneración en las vistas del cliente web.

Criterio	Puntos	Realizado
Reservaciones (Back End)	30%	30%
Integración Front/Back	25%	25%
Spring Security	25%	25%
Persistencia DB	10%	10%
Reporte y Documentación	10%	10%
Total	100%	100%

Table 2: Criterios de Evaluación Práctica 3

Luis David Martínez Martínez

9 Referencias Bibliográficas

References

- [Spring Security, 2026] Spring Projects. *Spring Security Reference Architecture Guide*. Broadcom. Disponible en:
<https://docs.spring.io/spring-security/reference/>
- [CORS MDN, 2026] MDN Web Docs. *Cross-Origin Resource Sharing (CORS)*. Mozilla Network. Disponible en:
<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>